

SATQUERY AI

P6 — Swarali

COMPLETE ENGINEERING PRD

Platform / DevOps + QA Engineer

Deadline: 20 September 2026 • Six developers in parallel • Contract-first microservices

This is an execution document, not an API-only specification. It defines ownership, repository locations, modules, issues, dependencies, tests, security, observability, daily execution and a ready-to-copy coding-agent prompt.

1. Role Mission

Field	Implementation specification
Primary owner	Docker, CI/CD, observability, test automation, security, release
Backup/review	Monitoring/alerts; release
Architecture reviewer	P1
QA/DevOps	P6
Success condition	Owned subsystem is independently runnable, contract-conformant, observable, secure and integrated into the flagship flood mission.
Non-ownership	Other teams' private tables, internal implementation details, credentials and undocumented contract changes.

2. System Position

P6 system position



Communication: browser→Gateway uses HTTPS/WebSocket; service-to-service uses versioned APIs/events; long-running EO/ML/report/monitoring work is asynchronous; storage and schemas have explicit ownership.

3. Repository / Folder Ownership

Folder	Purpose	Must not contain
infrastructure/docker/	Owned implementation, tests or docs	Unrelated service logic / credentials
infrastructure/monitoring/	Owned implementation, tests or docs	Unrelated service logic / credentials
infrastructure/release/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/contracts/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/integration/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/e2e/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/security/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/chaos/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/performance/	Owned implementation, tests or docs	Unrelated service logic / credentials

4. Technology Stack

Field	Implementation specification
Stack	Docker/GitHub Actions/OpenTelemetry/Prometheus/Grafana/pytest/Playwright/contract-security-load harnesses
Why	Matches the supplied SatQuery architecture and the six-person playbook; additions are implementation recommendations, not product requirements.
Serialization	OpenAPI/JSON Schema; canonical DTOs/events.
Testing	pytest and contract/integration tests; frontend uses browser E2E where applicable.
Observability	OpenTelemetry + structured logs + Prometheus/Grafana.
Containerization	Docker; Kubernetes remains future/post-SIH.

5. Engineering Rules

- All internal APIs use /api/v1; breaking changes require versioning and P1 review.
- Canonical ErrorResponse: code, message, details[], retryable, trace_id.
- Propagate trace_id, mission_id, run_id, job_id, organization_id where applicable.
- organization_id comes from verified auth context, never a request body.
- No cross-service private DB reads.

- Long-running EO/ML/report/monitoring work returns 202 + job_id.
- Results retain dataset_id/observation_ids/model_version/processing_version as applicable.
- Idempotency-Key is used for duplicate-sensitive writes.
- Remote calls use bounded timeouts/retries/backoff; no retry storms.
- Never log secrets, auth headers or raw sensitive inputs.

6. Complete Issue Register

ID	Title	Pri	Type	Depends	Contact
P6-01	Docker Compose local integration environment	P0	Infrastructure	P1-01	P1
P6-02	CI lint/type/unit pipeline	P0	CI	P1-01,P1-02	P5
P6-03	Security scanning baseline	P0	Security	P1-01,P1-02	P3
P6-04	Contract test harness and generated fixtures	P0	Testing	P6-03	P1
P6-05	DEV deployment and service health smoke	P0	DevOps	P6-04	P5
P6-06	OpenTelemetry collector and structured logging	P0	Observability	P6-05	P3
P6-07	Prometheus metrics and Grafana dashboards	P0	Observability	P6-06,P1-02	P1
P6-08	Cross-service integration test suite	P0	Testing	P6-07,P1-02	P5
P6-09	Flagship flood E2E automation	P0	E2E	P6-08,P1-02	P3
P6-10	Performance/load test harness	P1	Performance	P6-09,P1-02	P1
P6-11	Security/tenant/prompt-injection regression suite	P0	Security	P6-10	P5
P6-12	Failure injection and recovery tests	P0	Reliability	P6-11	P3
P6-13	Secrets/config and least-privilege hardening	P0	Security	P6-12	P1
P6-14	Monitoring/report integration QA	P0	Integration	P6-13	P5
P6-15	Release candidate, immutable images and rollback	P0	Release	P6-14,P5-05	P3
P6-16	Two full demo rehearsals and operator runbook	P0	Operations	P6-15,P5-05	P1
P6-17	Final QA sign-off and release gate	P0	Release	P6-16,P5-05	P5

P6-01 — Docker Compose local integration environment

Field	Implementation specification
Priority	P0
Type	Infrastructure
Owner	P6
Reviewer	P1
Cross-service contact	P1
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement docker compose local integration environment as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for docker compose local integration environment. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_docker_compose_local_integration_environment_valid()', 'test_docker_compose_local_integration_environment_invalid_input()']
Integration tests	['test_p6_01_service_boundary()']
Contract tests	['test_p6_01_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, docker compose local integration environment completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-02
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run docker compose local integration environment against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-02 — CI lint/type/unit pipeline

Field	Implementation specification
Priority	P0
Type	CI
Owner	P6
Reviewer	P1
Cross-service contact	P5
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement ci lint/type/unit pipeline as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for ci lint/type/unit pipeline. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_ci_lint_type_unit_pipeline_valid()', 'test_ci_lint_type_unit_pipeline_invalid_input()']
Integration tests	['test_p6_02_service_boundary()']
Contract tests	['test_p6_02_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, ci lint/type/unit pipeline completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-03
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run ci lint/type/unit pipeline against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-03 — Security scanning baseline

Field	Implementation specification
Priority	P0
Type	Security
Owner	P6
Reviewer	P1
Cross-service contact	P3
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement security scanning baseline as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for security scanning baseline. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_security_scanning_baseline_valid()', 'test_security_scanning_baseline_invalid_input()']
Integration tests	['test_p6_03_service_boundary()']
Contract tests	['test_p6_03_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, security scanning baseline completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-04
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run security scanning baseline against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-04 — Contract test harness and generated fixtures

Field	Implementation specification
Priority	P0
Type	Testing
Owner	P6
Reviewer	P1
Cross-service contact	P1
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement contract test harness and generated fixtures as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for contract test harness and generated fixtures. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_contract_test_harness_and_generated_fixtures_valid()', 'test_contract_test_harness_and_generated_fixtures_invalid_input()']
Integration tests	['test_p6_04_service_boundary()']
Contract tests	['test_p6_04_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, contract test harness and generated fixtures completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-03
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-05
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run contract test harness and generated fixtures against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-05 — DEV deployment and service health smoke

Field	Implementation specification
Priority	P0
Type	DevOps
Owner	P6
Reviewer	P1
Cross-service contact	P5
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement dev deployment and service health smoke as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for dev deployment and service health smoke. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_dev_deployment_and_service_health_smoke_valid()', 'test_dev_deployment_and_service_health_smoke_invalid_input()']
Integration tests	['test_p6_05_service_boundary()']
Contract tests	['test_p6_05_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, dev deployment and service health smoke completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-04
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-06
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run dev deployment and service health smoke against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-06 — OpenTelemetry collector and structured logging

Field	Implementation specification
Priority	P0
Type	Observability
Owner	P6
Reviewer	P1
Cross-service contact	P3
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement opentelemetry collector and structured logging as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for opentelemetry collector and structured logging. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_opentelemetry_collector_and_structured_logging_valid()', 'test_opentelemetry_collector_and_structured_logging_invalid_input()']
Integration tests	['test_p6_06_service_boundary()']
Contract tests	['test_p6_06_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, opentelemetry collector and structured logging completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-07
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run opentelemetry collector and structured logging against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-07 — Prometheus metrics and Grafana dashboards

Field	Implementation specification
Priority	P0
Type	Observability
Owner	P6
Reviewer	P1
Cross-service contact	P1
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement prometheus metrics and grafana dashboards as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for prometheus metrics and grafana dashboards. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_prometheus_metrics_and_grafana_dashboards_valid()', 'test_prometheus_metrics_and_grafana_dashboards_invalid_input()']
Integration tests	['test_p6_07_service_boundary()']
Contract tests	['test_p6_07_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, prometheus metrics and grafana dashboards completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-06,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-08
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run prometheus metrics and grafana dashboards against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-08 — Cross-service integration test suite

Field	Implementation specification
Priority	P0
Type	Testing
Owner	P6
Reviewer	P1
Cross-service contact	P5
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement cross-service integration test suite as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for cross-service integration test suite. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_cross_service_integration_test_suite_valid()', 'test_cross_service_integration_test_suite_invalid_input()']
Integration tests	['test_p6_08_service_boundary()']
Contract tests	['test_p6_08_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, cross-service integration test suite completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-07,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-09
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run cross-service integration test suite against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-09 — Flagship flood E2E automation

Field	Implementation specification
Priority	P0
Type	E2E
Owner	P6
Reviewer	P1
Cross-service contact	P3
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement flagship flood e2e automation as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for flagship flood e2e automation. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_flagship_flood_e2e_automation_valid()', 'test_flagship_flood_e2e_automation_invalid_input()']
Integration tests	['test_p6_09_service_boundary()']
Contract tests	['test_p6_09_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, flagship flood e2e automation completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-08, P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-10
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run flagship flood e2e automation against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-10 — Performance/load test harness

Field	Implementation specification
Priority	P1
Type	Performance
Owner	P6
Reviewer	P1
Cross-service contact	P1
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement performance/load test harness as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for performance/load test harness. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_performance_load_test_harness_valid()', 'test_performance_load_test_harness_invalid_input()']
Integration tests	['test_p6_10_service_boundary()']
Contract tests	['test_p6_10_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, performance/load test harness completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-09,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-11
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run performance/load test harness against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-11 — Security/tenant/prompt-injection regression suite

Field	Implementation specification
Priority	P0
Type	Security
Owner	P6
Reviewer	P1
Cross-service contact	P5
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement security/tenant/prompt-injection regression suite as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for security/tenant/prompt-injection regression suite. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_security_tenant_prompt_injection_regression_suite_valid()', 'test_security_tenant_prompt_injection_regression_suite_invalid_input()']
Integration tests	['test_p6_11_service_boundary()']
Contract tests	['test_p6_11_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, security/tenant/prompt-injection regression suite completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-10
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-12
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run security/tenant/prompt-injection regression suite against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-12 — Failure injection and recovery tests

Field	Implementation specification
Priority	P0
Type	Reliability
Owner	P6
Reviewer	P1
Cross-service contact	P3
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement failure injection and recovery tests as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for failure injection and recovery tests. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_failure_injection_and_recovery_tests_valid()', 'test_failure_injection_and_recovery_tests_invalid_input()']
Integration tests	['test_p6_12_service_boundary()']
Contract tests	['test_p6_12_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, failure injection and recovery tests completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-11
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-13
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run failure injection and recovery tests against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-13 — Secrets/config and least-privilege hardening

Field	Implementation specification
Priority	P0
Type	Security
Owner	P6
Reviewer	P1
Cross-service contact	P1
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement secrets/config and least-privilege hardening as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for secrets/config and least-privilege hardening. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_secrets_config_and_least_privilege_hardening_valid()', 'test_secrets_config_and_least_privilege_hardening_invalid_input()']
Integration tests	['test_p6_13_service_boundary()']
Contract tests	['test_p6_13_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, secrets/config and least-privilege hardening completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-12
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-14
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run secrets/config and least-privilege hardening against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-14 — Monitoring/report integration QA

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P6
Reviewer	P1
Cross-service contact	P5
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement monitoring/report integration qa as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for monitoring/report integration qa. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_monitoring_report_integration_qa_valid()', 'test_monitoring_report_integration_qa_invalid_input()']
Integration tests	['test_p6_14_service_boundary()']
Contract tests	['test_p6_14_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, monitoring/report integration qa completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-13
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-15
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run monitoring/report integration qa against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-15 — Release candidate, immutable images and rollback

Field	Implementation specification
Priority	P0
Type	Release
Owner	P6
Reviewer	P1
Cross-service contact	P3
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement release candidate, immutable images and rollback as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for release candidate, immutable images and rollback. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_release_candidate_immutable_images_and_rollback_valid()', 'test_release_candidate_immutable_images_and_rollback_invalid_input()']
Integration tests	['test_p6_15_service_boundary()']
Contract tests	['test_p6_15_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, release candidate, immutable images and rollback completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-14,P5-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-16
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run release candidate, immutable images and rollback against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-16 — Two full demo rehearsals and operator runbook

Field	Implementation specification
Priority	P0
Type	Operations
Owner	P6
Reviewer	P1
Cross-service contact	P1
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement two full demo rehearsals and operator runbook as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for two full demo rehearsals and operator runbook. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_two_full_demo_rehearsals_and_operator_runbook_valid()', 'test_two_full_demo_rehearsals_and_operator_runbook_invalid_input()']
Integration tests	['test_p6_16_service_boundary()']
Contract tests	['test_p6_16_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, two full demo rehearsals and operator runbook completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-15,P5-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P6-17
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run two full demo rehearsals and operator runbook against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P6-17 — Final QA sign-off and release gate

Field	Implementation specification
Priority	P0
Type	Release
Owner	P6
Reviewer	P1
Cross-service contact	P5
Service	Platform / QA / SRE
Folders	infrastructure/docker/; infrastructure/monitoring/; infrastructure/release/; tests/contracts/; tests/integration/; tests/e2e/; tests/security/; tests/chaos/; tests/performance/
Objective	Implement final qa sign-off and release gate as a production-shaped module inside the P6 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for final qa sign-off and release gate. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	infrastructure/docker/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	CI/CD, health, contract, integration, E2E, security, metrics and release automation
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_final_qa_sign_off_and_release_gate_valid()', 'test_final_qa_sign_off_and_release_gate_invalid_input()']
Integration tests	['test_p6_17_service_boundary()']
Contract tests	['test_p6_17_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, final qa sign-off and release gate completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P6-16, P5-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	Release/integration gate
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P6. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run final qa sign-off and release gate against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

7. Test Plan

The issue-level test names below are the minimum required suite. Expand edge cases discovered during implementation.

P6-01 — Docker Compose local integration environment

- `test_docker_compose_local_integration_environment_valid()`
- `test_docker_compose_local_integration_environment_invalid_input()`
- `test_p6_01_service_boundary()`
- `test_p6_01_schema_compatibility()`

P6-02 — CI lint/type/unit pipeline

- `test_ci_lint_type_unit_pipeline_valid()`
- `test_ci_lint_type_unit_pipeline_invalid_input()`
- `test_p6_02_service_boundary()`
- `test_p6_02_schema_compatibility()`

P6-03 — Security scanning baseline

- `test_security_scanning_baseline_valid()`
- `test_security_scanning_baseline_invalid_input()`
- `test_p6_03_service_boundary()`
- `test_p6_03_schema_compatibility()`

P6-04 — Contract test harness and generated fixtures

- `test_contract_test_harness_and_generated_fixtures_valid()`
- `test_contract_test_harness_and_generated_fixtures_invalid_input()`
- `test_p6_04_service_boundary()`
- `test_p6_04_schema_compatibility()`

P6-05 — DEV deployment and service health smoke

- `test_dev_deployment_and_service_health_smoke_valid()`
- `test_dev_deployment_and_service_health_smoke_invalid_input()`
- `test_p6_05_service_boundary()`
- `test_p6_05_schema_compatibility()`

P6-06 — OpenTelemetry collector and structured logging

- `test_opentelemetry_collector_and_structured_logging_valid()`
- `test_opentelemetry_collector_and_structured_logging_invalid_input()`
- `test_p6_06_service_boundary()`
- `test_p6_06_schema_compatibility()`

P6-07 — Prometheus metrics and Grafana dashboards

- `test_prometheus_metrics_and_grafana_dashboards_valid()`
- `test_prometheus_metrics_and_grafana_dashboards_invalid_input()`
- `test_p6_07_service_boundary()`
- `test_p6_07_schema_compatibility()`

P6-08 — Cross-service integration test suite

- `test_cross_service_integration_test_suite_valid()`
- `test_cross_service_integration_test_suite_invalid_input()`
- `test_p6_08_service_boundary()`
- `test_p6_08_schema_compatibility()`

P6-09 — Flagship flood E2E automation

- test_flagship_flood_e2e_automation_valid()
- test_flagship_flood_e2e_automation_invalid_input()
- test_p6_09_service_boundary()
- test_p6_09_schema_compatibility()

P6-10 — Performance/load test harness

- test_performance_load_test_harness_valid()
- test_performance_load_test_harness_invalid_input()
- test_p6_10_service_boundary()
- test_p6_10_schema_compatibility()

P6-11 — Security/tenant/prompt-injection regression suite

- test_security_tenant_prompt_injection_regression_suite_valid()
- test_security_tenant_prompt_injection_regression_suite_invalid_input()
- test_p6_11_service_boundary()
- test_p6_11_schema_compatibility()

P6-12 — Failure injection and recovery tests

- test_failure_injection_and_recovery_tests_valid()
- test_failure_injection_and_recovery_tests_invalid_input()
- test_p6_12_service_boundary()
- test_p6_12_schema_compatibility()

P6-13 — Secrets/config and least-privilege hardening

- test_secrets_config_and_least_privilege_hardening_valid()
- test_secrets_config_and_least_privilege_hardening_invalid_input()
- test_p6_13_service_boundary()
- test_p6_13_schema_compatibility()

P6-14 — Monitoring/report integration QA

- test_monitoring_report_integration_qa_valid()
- test_monitoring_report_integration_qa_invalid_input()
- test_p6_14_service_boundary()
- test_p6_14_schema_compatibility()

P6-15 — Release candidate, immutable images and rollback

- test_release_candidate_immutable_images_and_rollback_valid()
- test_release_candidate_immutable_images_and_rollback_invalid_input()
- test_p6_15_service_boundary()
- test_p6_15_schema_compatibility()

P6-16 — Two full demo rehearsals and operator runbook

- test_two_full_demo_rehearsals_and_operator_runbook_valid()
- test_two_full_demo_rehearsals_and_operator_runbook_invalid_input()
- test_p6_16_service_boundary()
- test_p6_16_schema_compatibility()

P6-17 — Final QA sign-off and release gate

- test_final_qa_sign_off_and_release_gate_valid()
- test_final_qa_sign_off_and_release_gate_invalid_input()
- test_p6_17_service_boundary()
- test_p6_17_schema_compatibility()

8. Security Implementation

Control	Implementation owner	Required behavior
Authentication	P6	Implement/test where applicable; P6 supplies regression gate.
RBAC + tenant isolation	P6	Implement/test where applicable; P6 supplies regression gate.
Input/schema validation	P6	Implement/test where applicable; P6 supplies regression gate.
GeoJSON/raster validation	P6	Implement/test where applicable; P6 supplies regression gate.
SSRF/provider allowlist	P6	Implement/test where applicable; P6 supplies regression gate.
Prompt injection	P6	Implement/test where applicable; P6 supplies regression gate.
AI tool permissions	P6	Implement/test where applicable; P6 supplies regression gate.
Rate/request limits	P6	Implement/test where applicable; P6 supplies regression gate.
Secrets/TLS	P6	Implement/test where applicable; P6 supplies regression gate.
Audit logging	P6	Implement/test where applicable; P6 supplies regression gate.
Dependency/container scanning	P6	Implement/test where applicable; P6 supplies regression gate.

9. Observability

Required identifiers: trace_id, mission_id, run_id, job_id, organization_id, model_version, dataset_id. Metrics should expose latency, queue time, errors and user-visible mission state. Role metrics: ci_duration_ms, contract_ci_fail_total, e2e_duration_ms, security_scan_fail_total

10. Failure Modes

Failure	Detection	Recovery/fallback	User behavior
Bhoonidhi unavailable	timeout/5xx	bounded retry → pinned permitted fixture	show fallback clearly
Inference timeout/OOM	job timeout/worker exit	bounded retry → deterministic fixture for demo	show explicit degraded/failure state
Redis restart	connection/requeue failure	idempotent requeue/reconciliation	mission remains recoverable
Invalid AOI/raster	validator rejection	no expensive processing	show actionable validation
Low confidence	confidence gate	clarify/acquire/explicit uncertainty	never confident prose
Tenant violation	RBAC denial	no retry/no leakage	403/404

11. Implementation Order

Foundation → core domain → cross-service integration → differentiation → hardening/release. Earlier issues unlock later ones, but frozen contracts and fixtures allow parallel implementation. Do not serialize on live dependencies. Use mocks whenever the producer is unfinished.

12. Daily Execution Plan — Sep 11–20

Date	Issues	Expected output	Dependency	Integration checkpoint	Tests/demo
Sep 11	P6-01, P6-02	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 12	P6-02, P6-03	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 13	P6-03, P6-04	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 14	P6-04, P6-05	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 15	P6-05, P6-06	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 16	P6-06, P6-07	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 17	P6-07, P6-08	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 18	P6-08, P6-09	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 19	P6-09, P6-10	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 20	P6-10, P6-11	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state

13. Definition of Ready / Done

Field	Implementation specification
DoR	Requirements clear; owner/reviewer; contract; dependencies; acceptance; tests; demo result defined.
DoD	Merged; unit/contract/integration tests; security; telemetry; docs/schema; DEV/DEMO; acceptance demonstrated.

14. Cross-Team Communication

- P1 is architecture/contract authority.
- P6 is CI/DEV/DEMO/release authority.
- P2↔P3: sensor arbitration, inference and evidence.
- P2↔P4: observation selection, data quality and acquisition.
- P3↔P4: AssetRef, CRS/resolution and preprocessing.
- P4↔P5: tiles, masks, GeoJSON and map performance.
- P5↔P1: generated client and browser DTOs.
- Escalate blockers after 30–60 minutes; use contract mocks/fixtures instead of waiting.

15. Prompt to Give to the Coding Agent

You are P6 implementation agent. Own Docker/Compose, CI/CD, observability, integration/E2E/security/performance/chaos and release. Make every service reproducible and health-checkable. Gate lint/type/unit/contract/security/container/integration/E2E/performance as appropriate. Trace user→mission→run→agent→tool→observation→geo→inference→evidence→report. Maintain deterministic flood E2E, failure profiles, immutable RC and rollback. Never bypass critical security/E2E gates.

16. Role Release Checklist

- ☐ All P0 issues merged or explicitly waived.
- ☐ Unit + contract + relevant integration tests green.
- ☐ Security controls and negative tests green.
- ☐ Trace/metrics/logging present.
- ☐ DEV/DEMO verified.
- ☐ Failure behavior demonstrated.
- ☐ Fixtures/model versions pinned where applicable.
- ☐ Documentation/ADR updated.
- ☐ Role-specific flagship demo path works.